Optimization Algorithm Course, A.Y. 2024/2025

Tabu Search for solving the Single-Batch Processing Machine Scheduling Problem

based on: *A variant for the Single-Batch Processing Machine Scheduling Problem* by *A. Castelletti, R. Mansini, L. Moreschini*

Luca Pugnetti

UNIVERSITÀ DEGLI STUDI DI BRESCIA

Single Batch-Processing Machine Scheduling Problem

Data:

- A set of jobs. Each job has:
  - size,
  - processing time,
  - family.
- Machine capacity, that each batch (group of jobs) must satisfy.
- Setup times for dominant families of consecutive batches.

Batch Processing:

- Processing time of a batch = job with the highest processing time in that batch.
- Dominant family = family with highest cooking temperature among jobs in that batch.

Objective (Makespan):

$$\min \left(\sum_{\text{batches}} \text{processing time} \quad + \sum_{\text{consecutive batches}} \text{setup time} \right)$$

Overview of the Tabu Search Algorithm

Tabu Search is a metaheuristic that efficiently explores the solution space by using *memory structures* which prevent revisiting previously explored solutions.

- **Iterative improvement**: At each iteration, the algorithm searches a neighborhood of the current solution.
- **Feasibility and tabu restrictions**: It selects the best neighbor that is not tabu, or meets the *aspiration* criterion if it is tabu.
- **Stopping condition**: The process continues until the *time limit* is reached (10 minutes per benchmark instance).

Initialization Heuristics for Tabu Search

Three heuristics are considered for initial solution construction (the one with the lowest makespan is chosen):

- **Ascending Order:** Jobs sorted in ascending order by size, assigned sequentially to batches until machine capacity is reached.
- **Descending Order:** Jobs sorted in descending order and assigned similarly.
- **Family-Based Assignment:** Jobs grouped by family, forming batches to minimize setup times.

Initialization Heuristics: Example

Given:

- Job sizes: $\{8, 3, 5, 2, 6, 4, 7, 1\}$
- Machine capacity: 10
- Job families:
  - Family A: $\{1, 8\}$
  - Family B: $\{3, 6\}$
  - Family C: $\{2, 4\}$
  - Family D: $\{5, 7\}$

Heuristic	**Batch 1**	**Batch 2**	**Batch 3**	**Batch 4**	**Batch 5**
Ascending Order	$\{1, 2, 3, 4\}$	$\{5\}$	$\{6\}$	$\{7\}$	$\{8\}$
Descending Order	$\{8\}$	$\{7\}$	$\{6\}$	$\{5, 4\}$	$\{3, 2, 1\}$
Family-Based	$\{1, 8\}$	$\{3, 6\}$	$\{2, 4\}$	$\{5\}$	$\{7\}$

Makespan Calculation

Mathematically, the makespan is given by:

$$M = \sum_{b=1}^{B} \max_{j \in B_b} P_j + \sum_{b=2}^{B} S(F(B_{b-1}), F(B_b))$$

where:

- B is the total number of batches,
- B_b represents batch b,
- P_j is the processing time of job j,
- $\max_{j \in B_b} P_j$ is the processing time of batch b,
- $F(B_b)$ is the dominant job family in batch b,
- $S(F(B_{b-1}), F(B_b))$ is the setup time between two consecutive batches.

Penalized Makespan Calculation

The wasted capacity is defined as:

$$W = \sum_{b=1}^{B} \left(C - \sum_{j \in B_b} S_j \right)$$

where:

- C is the total machine capacity.
- S_j is the size of job j.

The final penalized score is computed as:

$$M_p = M + \lambda W \tag{1}$$

where:

- λ (PENALIZATION_FACTOR) is a small constant,
- M is the makespan computed as before.

Tabu Tenure

In this implementation, the tabu tenure is dynamically set based on the instance size (number of jobs). Specifically, it is computed as:

$$\text{TABU_TENURE} = \left\lfloor \frac{n_{\text{jobs}}}{2} \right\rfloor$$

where:

- n_{jobs} is the total number of jobs in the instance.

Aspiration Criteria

In Tabu Search, a tabu solution may still be accepted under two conditions:

1. **A Better Tabu Solution Exists:** If a tabu solution S_{new} improves the best makespan found so far, it is accepted despite being tabu.
2. **All Neighbors Are Tabu:** If all candidate solutions in the neighborhood $\mathcal{N}(S^*)$ are tabu, the algorithm selects at least one to continue the search, avoiding stagnation.

Restart Threshold, for avoiding stagnation

The restart threshold is defined as:

$$\text{RESTART_THRESHOLD} = \left\lfloor \frac{n_{\text{jobs}}}{2} \cdot 10 \right\rfloor$$

where:

- n_{jobs} is the total number of jobs in the instance.

If no better solution is found after this many consecutive iterations, the algorithm restructures the batch assignments by randomly shuffling job allocations while maintaining capacity feasibility.

Tabu Search for SBPMSP Algorithm

1. **Initialize:**
   - Identify a feasible solution $\hat{x}$ (above the three heuristics).
   - Set the current solution $x \leftarrow \hat{x}$.
   - Initialize the tabu list $TL \leftarrow \varnothing$.
2. **Generate Neighborhood:**
   - Construct the neighborhood $\mathcal{N}(x)$ of x.
3. **Select the Best Neighbor:**
   - Find a solution $x' \in \mathcal{N}(x)$ such that:
     - $f(x')$ is minimal (best objective value among neighbors),
     - x' is not tabu (or satisfies aspiration).
4. **Update Current Solution:**
   - Set $x \leftarrow x'$.
   - Insert the move leading from x to x' into the tabu list TL.
5. **Check Stopping Criterion:**
   - If stopping condition (time limit) is not met return to Step 2 (if restart treshold reached, perturbate current solution x).
   - Otherwise, **stop** and return the current solution x.

Instances Characteristics

Each instance is defined by the following key parameters:

- **Number of Jobs (n)**: Total jobs per instance, ranging from 8 to 100.
- **Number of Families (F)**: Jobs are grouped into 4 to 10 families, affecting setup times.
- **Batch Capacity (C)**: Maximum allowed batch capacity per instance.

Instance	**Jobs (n)**	**Families (F)**	**Capacity (C)**
Example	8	4	150
50_5_100-0	50	5	100
50_5_100-1	50	5	100
50_5_100-4	50	5	100
50_5_100-6	50	5	100
50_5_100-8	50	5	100
100_10_200-10	100	10	200
100_10_200-11	100	10	200
100_10_200-14	100	10	200
100_10_200-15	100	10	200
100_10_200-18	100	10	200

Tabu Search vs. Gurobi Results

The performance of the Tabu Search algorithm is compared to Gurobi (1-hour execution). The key metrics analyzed include:

- **Makespan**: Completion time of the last batch.
- **Gap (%)**: Relative difference between Tabu and Gurobi solutions.

Instance	Gurobi (1h)	Tabu Search	Gap (%)
Example	70	70	0.00
50_5_100-0	279	279	0.00
50_5_100-1	294	294	0.00
50_5_100-4	285	285	0.00
50_5_100-6	326	326	0.00
50_5_100-8	288	289	0.35
100_10_200-10	2815	2947	4.69
100_10_200-11	2688	2746	2.16
100_10_200-14	2690	2779	3.31
100_10_200-15	2805	2852	1.68
100_10_200-18	2665	2720	2.06